

Dart Algorithms & Data Structures Master

Candidate: Irshath Ahamed | Senior Flutter Developer (Portal/Mobile)

Dart Data Structures, Algorithms & Logic — Interview Master Workbook

This workbook contains the **12 most frequently asked logic, data structure, and algorithmic problems** in Flutter/Dart technical coding rounds. Every solution is written in **idiomatic Dart**, complete with **Time & Space Complexity analysis**, and an explanation of the underlying logic.

Problem Index

1. [Problem 1: Merge Overlapping Shift Intervals \(Real-World Timesheet Logic\)](#)
2. [Problem 2: Two Sum with \$O\(N\)\$ Hash Map Lookup](#)
3. [Problem 3: Longest Substring Without Repeating Characters \(Sliding Window\)](#)
4. [Problem 4: True \$O\(1\)\$ LRU Cache \(HashMap + Doubly Linked List\)](#)
5. [Problem 5: Valid Parentheses / Nested Tag Matching \(Stack\)](#)
6. [Problem 6: Flatten Hierarchical Organization Tree to Indented List](#)
7. [Problem 7: Binary Search in Rotated Sorted Array](#)
8. [Problem 8: Group Anagrams \(\$O\(N \cdot K\)\$ Hashing\)](#)
9. [Problem 9: In-Place 90-Degree Matrix Rotation \(Image Processing Logic\)](#)
10. [Problem 10: Token Bucket Rate Limiter \(Network Throttling Logic\)](#)
11. [Problem 11: Lowest Common Ancestor \(Tree Traversal — Widget Tree Analogy\)](#)
12. [Problem 12: Deep Object Diffing Algorithm \(State Change Detection\)](#)

Problem 1: Merge Overlapping Shift Intervals

Problem Statement:

Given a list of work shift intervals `[start, end]`, merge all overlapping intervals into contiguous time blocks. (Crucial for attendance systems where employees have overlapping break or site visit logs).

- **Example Input:** `[[1, 3], [2, 6], [8, 10], [15, 18]]`
- **Output:** `[[1, 6], [8, 10], [15, 18]]`
- **Complexity:** Time: $O(N \log N)$ (sorting) | Space: $O(N)$

```

class Interval {
    final int start;
    final int end;
    const Interval(this.start, this.end);

    @override
    String toString() => '[$start, $end]';
}

List<Interval> mergeShiftIntervals(List<Interval> intervals) {
    if (intervals.length <= 1) return intervals;

    // 1. Sort intervals by start time ascending
    final sorted = List<Interval>.from(intervals)
        ..sort((a, b) => a.start.compareTo(b.start));

    final List<Interval> merged = [];
    Interval current = sorted.first;

    for (int i = 1; i < sorted.length; i++) {
        final next = sorted[i];

        if (next.start <= current.end) {
            // Overlap detected: extend current end to max of both
            current = Interval(current.start, current.end > next.end ? current.end : next.end);
        } else {
            // No overlap: push current and start new interval
            merged.add(current);
            current = next;
        }
    }

    merged.add(current);
    return merged;
}

```

Problem 2: Two Sum with $O(N)$ Hash Map Lookup

Problem Statement:

Given an array of integers `nums` and an integer `target`, return the **indices** of the two numbers such that they add up to `target`.

- **Complexity:** Time: $O(N)$ | Space: $O(N)$

```

List<int> twoSum(List<int> nums, int target) {
    // Map stores: value -> index
    final Map<int, int> seen = {};

    for (int i = 0; i < nums.length; i++) {
        final current = nums[i];
        final complement = target - current;

        if (seen.containsKey(complement)) {
            return [seen[complement]!, i];
        }

        seen[current] = i;
    }

    return []; // No pair found
}

```

Problem 3: Longest Substring Without Repeating Characters

Problem Statement:

Given a string `s`, find the length of the longest substring without repeating characters using the **Sliding Window** technique.

- **Example:** `"abcabcbb"` $\rightarrow$ `3` (`"abc"`)
- **Complexity:** Time: $O(N)$ | Space: $O(\min(N, \text{Alphabet Size}))$

```

import 'dart:math';

int lengthOfLongestSubstring(String s) {
    final Map<String, int> charLastIndex = {};
    int maxLength = 0;
    int windowStart = 0;

    for (int windowEnd = 0; windowEnd < s.length; windowEnd++) {
        final char = s[windowEnd];

        // If character was seen inside current window, move windowStart past its last position
        if (charLastIndex.containsKey(char) && charLastIndex[char]! >= windowStart) {
            windowStart = charLastIndex[char]! + 1;
        }

        charLastIndex[char] = windowEnd;
        maxLength = max(maxLength, windowEnd - windowStart + 1);
    }

    return maxLength;
}

```

Problem 4: True $O(1)$ LRU Cache (HashMap + Doubly Linked List)

Problem Statement:

Design an in-memory **LRU (Least Recently Used) Cache** where both `get(key)` and `put(key, value)` run in strict **$O(1)$ time complexity**.

- **Complexity:** Time: $O(1)$ for both operations | Space: $O(\text{Capacity})$

```

class _Node<K, V> {
    K key;
    V value;
    _Node<K, V>? prev;
    _Node<K, V>? next;

    _Node(this.key, this.value);
}

class LruCache<K, V> {
    final int capacity;
    final Map<K, _Node<K, V>> _map = {};
    late _Node<K, V> _head;
    late _Node<K, V> _tail;

    LruCache(this.capacity) {
        // Dummy sentinel nodes to avoid null checks
        _head = _Node<K, V>(null as dynamic, null as dynamic);
        _tail = _Node<K, V>(null as dynamic, null as dynamic);
        _head.next = _tail;
        _tail.prev = _head;
    }

    V? get(K key) {
        final node = _map[key];
        if (node == null) return null;

        // Move accessed node to head (most recently used)
        _remove(node);
        _insertAtHead(node);
        return node.value;
    }

    void put(K key, V value) {
        if (_map.containsKey(key)) {
            final existing = _map[key]!;
            existing.value = value;
            _remove(existing);
            _insertAtHead(existing);
            return;
        }

        if (_map.length >= capacity) {
            // Evict least recently used (node right before tail)
            final lru = _tail.prev!;
            _remove(lru);
            _map.remove(lru.key);
        }

        final newNode = _Node<K, V>(key, value);
        _insertAtHead(newNode);
        _map[key] = newNode;
    }

    void _remove(_Node<K, V> node) {
        node.prev?.next = node.next;
        node.next?.prev = node.prev;
    }
}

```

```

void _insertAtHead(_Node<K, V> node) {
    node.next = _head.next;
    node.prev = _head;
    _head.next?.prev = node;
    _head.next = node;
}
}

```

Problem 5: Valid Parentheses / Nested Tag Matching

Problem Statement:

Determine if an input string containing `()`, `{}`, and `[]` is valid (every open bracket is closed by the same type in correct order).

- **Complexity:** Time: $O(N)$ | Space: $O(N)$

```

bool isValidParentheses(String s) {
    final List<String> stack = [];
    final Map<String, String> matchingPairs = {
        ')': '(',
        '}': '{',
        ']': '[',
    };

    for (int i = 0; i < s.length; i++) {
        final char = s[i];

        if (matchingPairs.containsKey(char)) {
            // Closing bracket: top of stack must match
            if (stack.isEmpty || stack.removeLast() != matchingPairs[char]) {
                return false;
            }
        } else {
            // Opening bracket
            stack.add(char);
        }
    }

    return stack.isEmpty;
}

```

Problem 6: Flatten Hierarchical Organization Tree to Indented List

Problem Statement:

Convert a nested Employee Organizational Tree into a flat list with indentation levels suitable for rendering in an adaptive directory list.

- **Complexity:** Time: $O(N)$ | Space: $O(\text{Depth})$ recursion stack

```

class OrgNode {
    final String name;
    final String designation;
    final List<OrgNode> subordinates;

    const OrgNode({
        required this.name,
        required this.designation,
        this.subordinates = const [],
    });
}

class FlatOrgItem {
    final String name;
    final String designation;
    final int depthLevel; // 0 = CEO, 1 = Manager, 2 = Team Lead

    const FlatOrgItem(this.name, this.designation, this.depthLevel);

    @override
    String toString() => '{$" " * depthLevel}• $name ($designation)';
}

List<FlatOrgItem> flattenOrganizationTree(OrgNode root) {
    final List<FlatOrgItem> result = [];

    void traverse(OrgNode node, int depth) {
        result.add(FlatOrgItem(node.name, node.designation, depth));
        for (final child in node.subordinates) {
            traverse(child, depth + 1);
        }
    }

    traverse(root, 0);
    return result;
}

```

Problem 7: Binary Search in Rotated Sorted Array

Problem Statement:

Given a sorted array rotated at an unknown pivot (e.g., `[4, 5, 6, 7, 0, 1, 2]`), find the index of a `target` in strict $O(\log N)$ time.

- **Complexity:** Time: $O(\log N)$ | Space: $O(1)$

```

int searchInRotatedArray(List<int> nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        final mid = left + ((right - left) >> 1); // Bit shift division by 2

        if (nums[mid] == target) return mid;

        // Check if left half is normally sorted
        if (nums[left] <= nums[mid]) {
            if (target >= nums[left] && target < nums[mid]) {
                right = mid - 1; // Target lies in left half
            } else {
                left = mid + 1; // Target lies in right half
            }
        } else {
            // Right half must be sorted
            if (target > nums[mid] && target <= nums[right]) {
                left = mid + 1; // Target lies in right half
            } else {
                right = mid - 1; // Target lies in left half
            }
        }
    }

    return -1; // Target not found
}

```

Problem 8: Group Anagrams ($O(N \cdot K)$ Hashing)

Problem Statement:

Group words that are anagrams of each other.

- **Example:** ["eat", "tea", "tan", "ate", "nat", "bat"]
- **Output:** [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
- **Complexity:** Time: $O(N \cdot K \log K)$ | Space: $O(N \cdot K)$

```

List<List<String>> groupAnagrams(List<String> words) {
    final Map<String, List<String>> groups = {};

    for (final word in words) {
        // Sort characters of the word to form the canonical key
        final sortedChars = word.split('').sort();
        final key = sortedChars.join();

        groups.putIfAbsent(key, () => []).add(word);
    }

    return groups.values.toList();
}

```

Problem 9: In-Place 90-Degree Matrix Rotation

Problem Statement:

Rotate an $N \times N$ 2D matrix clockwise by 90 degrees in-place without allocating a second matrix. (Analogous to image orientation transformations in custom photo pickers).

- **Logic:** 1. Transpose the matrix (swap `matrix[i][j]` with `matrix[j][i]`), 2. Reverse each row.
- **Complexity:** Time: $O(N^2)$ | Space: $O(1)$ in-place

```
void rotateMatrix90Clockwise(List<List<int>> matrix) {
    final n = matrix.length;

    // 1. Transpose Matrix
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            final temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }

    // 2. Reverse each row
    for (int i = 0; i < n; i++) {
        int left = 0;
        int right = n - 1;
        while (left < right) {
            final temp = matrix[i][left];
            matrix[i][left] = matrix[i][right];
            matrix[i][right] = temp;
            left++;
            right--;
        }
    }
}
```

Problem 10: Token Bucket Rate Limiter

Problem Statement:

Implement the **Token Bucket Algorithm** to throttle outgoing network requests or check-in attempts on client devices.

- **Complexity:** Time: $O(1)$ check | Space: $O(1)$

```

import 'dart:math';

class TokenBucketRateLimiter {
  final double maxTokens;
  final double refillRatePerSecond;

  double _currentTokens;
  DateTime _lastRefillTimestamp;

  TokenBucketRateLimiter({
    required this.maxTokens,
    required this.refillRatePerSecond,
  }) : _currentTokens = maxTokens,
       _lastRefillTimestamp = DateTime.now();

  bool tryConsume([double tokens = 1.0]) {
    _refill();

    if (_currentTokens >= tokens) {
      _currentTokens -= tokens;
      return true; // Request allowed
    }

    return false; // Rate limited
  }

  void _refill() {
    final now = DateTime.now();
    final elapsedSeconds = now.difference(_lastRefillTimestamp).inMilliseconds / 1000.0;
    _lastRefillTimestamp = now;

    // Replenish tokens based on elapsed time up to max capacity
    _currentTokens = min(maxTokens, _currentTokens + (elapsedSeconds * refillRatePerSecond));
  }
}

```

Problem 11: Lowest Common Ancestor (Tree Traversal)

Problem Statement:

Find the Lowest Common Ancestor (LCA) of two nodes **p** and **q** in a Binary Tree. *(Directly mirrors how Flutter finds the closest shared parent in the Element Tree when dispatching notifications).*

- **Complexity:** Time: $O(N)$ | Space: $O(H)$ where H is tree height

```

class TreeNode {
    final int val;
    TreeNode? left;
    TreeNode? right;
    TreeNode(this.val);
}

TreeNode? findLowestCommonAncestor(TreeNode? root, TreeNode p, TreeNode q) {
    if (root == null || root.val == p.val || root.val == q.val) {
        return root;
    }

    final left = findLowestCommonAncestor(root.left, p, q);
    final right = findLowestCommonAncestor(root.right, p, q);

    // If both left and right return non-null, root is the LCA
    if (left != null && right != null) return root;

    // Otherwise return whichever subtree found a match
    return left ?? right;
}

```

Problem 12: Deep Object Diffing Algorithm

Problem Statement:

Write a recursive function that compares two nested JSON/Map objects and outputs a detailed list of added, removed, and modified keys. (*Crucial for optimistic offline synchronization and dirty-state tracking*).

- **Complexity:** Time: $O(\text{Total Nodes})$ | Space: $O(\text{Depth})$

```

enum DiffType { added, removed, modified }

class FieldDiff {
    final String path;
    final DiffType type;
    final dynamic oldValue;
    final dynamic newValue;

    const FieldDiff(this.path, this.type, this.oldValue, this.newValue);

    @override
    String toString() => '[$type] $path: $oldValue -> $newValue';
}

List<FieldDiff> diffJson(Map<String, dynamic> oldMap, Map<String, dynamic> newMap, [String currentPath]) {
    final List<FieldDiff> diffs = [];
    final allKeys = {...oldMap.keys, ...newMap.keys};

    for (final key in allKeys) {
        final fieldPath = currentPath.isEmpty ? key : '$currentPath.$key';
        final hasOld = oldMap.containsKey(key);
        final hasNew = newMap.containsKey(key);

        if (!hasOld && hasNew) {
            diffs.add(FieldDiff(fieldPath, DiffType.added, null, newMap[key]));
        } else if (hasOld && !hasNew) {
            diffs.add(FieldDiff(fieldPath, DiffType.removed, oldMap[key], null));
        } else {
            final oldVal = oldMap[key];
            final newVal = newMap[key];

            if (oldVal is Map<String, dynamic> && newVal is Map<String, dynamic>) {
                // Recursively diff nested maps
                diffs.addAll(diffJson(oldVal, newVal, fieldPath));
            } else if (oldVal != newVal) {
                diffs.add(FieldDiff(fieldPath, DiffType.modified, oldVal, newVal));
            }
        }
    }

    return diffs;
}

```